Setup Guide Steps

Run the Recruitment application on your own computer

If another developer receives your complete project files (or clones your GitHub repository) and installs all the required software, they must configure their own database, build the backend, install the frontend dependencies, and run both applications.

They do not need to use your laptop or your local MySQL database. They can run the same application on their own computer with their own database.

What the other developer should do

The order is important. Follow these steps:

1 Get your project code

They can download the ZIP file or clone your GitHub repository.

```
git clone https://github.com/naveentechsavvy/recruitment.git
cd recruitment
git checkout develop
```

2 Create their own MySQL database

Open MySQL Workbench and connect to their local MySQL server.

Run:

```
CREATE DATABASE recruitment_db
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;
```

They should create their own database username and password:

```
CREATE USER 'recruitment_app'@'localhost'
IDENTIFIED BY 'TheirOwnPassword';

GRANT ALL PRIVILEGES ON recruitment_db.*
TO 'recruitment_app'@'localhost';

FLUSH PRIVILEGES;
```

This database will be empty initially. Your existing candidates, clients, users, and other records will not automatically be copied to their computer.

3 Configure the database connection

Open PowerShell inside the **recruitment-backend** folder and enter:

```
$env:DB_HOST = 'localhost'
$env:DB_PORT = '3306'
$env:DB_NAME = 'recruitment_db'
$env:DB_USERNAME = 'recruitment_app'
$env:DB_PASSWORD = 'TheirOwnPassword'
```

These values connect the application to their own MySQL database. They should use the password they created in Step 2.

4 Build and run the backend

In the same PowerShell window, while inside **recruitment-backend**, run:

```
mvn clean install
```

This builds the Java backend and checks for compilation errors.

After it succeeds, run:

```
mvn spring-boot:run "-Dspring-boot.run.profiles=local"
```

The backend should start on: **http://localhost:8080**

5 Install and run the frontend

Open a second PowerShell window and go to the **recruitment-admin** folder:

```
cd path\to\recruitment\recruitment-admin
```

Install the frontend dependencies:

```
npm ci
```

If the project does not contain package-lock.json, use **npm install** instead.

Start Angular:

```
npx ng serve
```

The frontend should start on: **http://localhost:4200**

Keep both PowerShell windows open while using the application.

6 First-time admin login

Before the first backend startup, they should set their own admin credentials in the backend PowerShell window:

```
$env:ADMIN_EMAIL = 'their-email@example.com'
$env:ADMIN_PASSWORD = 'TheirStrongPassword'
```

The application creates the initial admin account automatically if it does not already exist. They should configure these values before the first startup. Changing them later does not automatically change an existing admin account's password.

Final order — what they need to do

1	**Install required software** Java 17, Maven, Node.js, Angular dependencies and MySQL.
2	**Get your project code** Download ZIP or clone your GitHub repository.
3	**Set up MySQL** Create their own database and database user.
4	**Configure credentials** Set their own database username/password and admin credentials.
5	**Build and run backend** mvn clean install, then mvn spring-boot:run.
6	**Install and run frontend** npm ci, then npx ng serve.
7	**Open the application** Visit http://localhost:4200.

If they want to use your existing candidates, clients, users and other records, you must separately provide a MySQL database backup and any required uploaded files. Otherwise, they start with a fresh database.

One more thing: Email functionality requires their own SMTP configuration if they want invitation emails and offer letters to work. They should also configure a proper JWT secret rather than use the development default.

> **In short:** Yes, they build the backend, run it, install and run the frontend, and connect it to their own MySQL database. They do not need to manually build all the SQL tables.

Email password setup (Gmail)

If the developer wants invitation emails, offer letters and other email notifications to work, they must configure their own Gmail account. The emails will then be sent from their own email address.

1 Create a Gmail App Password

1. Open Google Account (**https://myaccount.google.com/**).
2. Go to **Security**.
3. Enable **2-Step Verification**.
4. Search for **App passwords**.
5. Create a new App Password.
6. Copy the generated 16-character password.

Note: App Passwords are different from the normal Gmail login password.

2 Configure email in PowerShell

Open PowerShell inside the **recruitment-backend** folder and enter:

```
$env:MAIL_HOST = 'smtp.gmail.com'
$env:MAIL_PORT = '587'
$env:MAIL_USERNAME = 'your-email@gmail.com'
$env:MAIL_PASSWORD = 'your-16-character-app-password'
$env:MAIL_SMTP_AUTH = 'true'
$env:MAIL_SMTP_STARTTLS = 'true'
$env:MAIL_FROM = 'your-email@gmail.com'
```

Replace the example email and password with their own Gmail details.

3 Run the backend

In the same PowerShell window:

```
mvn spring-boot:run "-Dspring-boot.run.profiles=local"
```

Important

- Every developer should configure their own email account and App Password.
- Do not share your personal Gmail password or App Password.
- These environment variables apply only to the current PowerShell window.
- If they close PowerShell, they need to set them again next time.

> That's all for Gmail email configuration.